

24CSI024-DATA STRUCTURES AND ALGORITHMS

UNIT 2 LINEAR DATA STRUCTURES

STACK AND QUEUE

Pre Lab Task

Learn the following constructs:

1. Loops
2. Conditional Statements
3. Functions
4. File Handling

In Lab exercises

1. Implement

CODE:

```
1 #include <stdio.h>
2 #include <ctype.h>
3 #define MAX 100
4 char charStack[MAX];
5 int charTop = -1;
6 int intStack[MAX];
7 int intTop = -1;
8 void pushChar(char x) {
9     charStack[++charTop] = x;
10 }
11 char popChar() {
12     return charStack[charTop--];
13 }
14 void pushInt(int x) {
15     intStack[++intTop] = x;
16 }
17 int popInt() {
18     return intStack[intTop--];
19 }
20 int precedence(char x) {
21     if (x == '^')
22         return 3;
23     if (x == '*' || x == '/')
24         return 2;
25     if (x == '+' || x == '-')
26         return 1;
27     return 0;
28 }
29 void infixToPostfix() {
30     char infix[MAX], postfix[MAX];
```

```
31     int i, j = 0;
32     charTop = -1;
33     printf("\nEnter an infix expression: ");
34     scanf("%s", infix);
35     for (i = 0; infix[i] != '\0'; i++) {
36         if (isdigit(infix[i])) {
37             postfix[j++] = infix[i];
38         }
39         else if (infix[i] == '(') {
40             pushChar(infix[i]);
41         }
42         else if (infix[i] == ')') {
43             while (charTop != -1 && charStack[charTop] != '(')
44                 postfix[j++] = popChar();
45             if (charTop != -1)
46                 popChar();
47         }
48         else {
49             while (charTop != -1 &&
50                 charStack[charTop] != '(' &&
51                 precedence(charStack[charTop]) >= precedence(infix[i])) {
52                 postfix[j++] = popChar();
53             }
54             pushChar(infix[i]);
55         }
56     }
57     while (charTop != -1)
58         postfix[j++] = popChar();
59     postfix[j] = '\0';
60     printf("Postfix expression: %s\n", postfix);
```

```

60     printf("Postfix expression: %s\n", postfix);
61 }
62 void evaluatePostfix() {
63     char exp[MAX];
64     int i, a, b, result;
65     intTop = -1;
66     printf("\nEnter a postfix expression: ");
67     scanf("%s", exp);
68     for (i = 0; exp[i] != '\0'; i++) {
69         if (isdigit(exp[i])) {
70             pushInt(exp[i] - '0');
71         }
72         else {
73             if (intTop < 1) {
74                 printf("Invalid expression\n");
75                 return;
76             }
77             b = popInt();
78             a = popInt();
79             switch (exp[i]) {
80                 case '+':
81                     result = a + b;
82                     break;
83                 case '-':
84                     result = a - b;
85                     break;
86                 case '*':
87                     result = a * b;
88                     break;

```

```

89                 case '/':
90                     if (b == 0) {
91                         printf("Division by zero is not allowed\n");
92                         return;
93                     }
94                     result = a / b;
95                     break;
96                 case '^':
97                     result = 1;
98                     for (int j = 0; j < b; j++)
99                         result *= a;
100                     break;
101                 default:
102                     printf("Invalid operator\n");
103                     return;
104             }
105             pushInt(result);
106         }
107     }
108     if (intTop != 0) {
109         printf("Invalid expression\n");
110         return;
111     }
112     printf("Result: %d\n", popInt());
113 }

```

```

114 void parenthesisMatching() {
115     char exp[MAX];
116     int i, valid = 1;
117     charTop = -1;
118     printf("\nEnter an expression: ");
119     scanf("%s", exp);
120     for (i = 0; exp[i] != '\0'; i++) {
121         if (exp[i] == '(' || exp[i] == '[' || exp[i] == '{') {
122             pushChar(exp[i]);
123         }
124         else if (exp[i] == ')' || exp[i] == ']' || exp[i] == '}') {
125             if (charTop == -1) {
126                 valid = 0;
127                 break;
128             }
129             char x = popChar();
130             if ((exp[i] == ')') && x != '(') ||
131                 (exp[i] == ']') && x != '[') ||
132                 (exp[i] == '}') && x != '{') {
133                 valid = 0;
134                 break;
135             }
136         }
137     }
138     if (charTop != -1)
139         valid = 0;
140     if (valid)
141         printf("Parentheses are balanced\n");
142     else
143         printf("Parentheses are not balanced\n");

```

```

142     else
143     }
144 }
145 int main()
146 {
147     int choice;
148     while (1) {
149         printf("\n==== STACK APPLICATIONS =====\n");
150         printf("1. Infix to Postfix\n");
151         printf("2. Postfix Expression Evaluation\n");
152         printf("3. Parenthesis Matching\n");
153         printf("4. Exit\n");
154         printf("Enter your choice: ");
155         scanf("%d", &choice);
156         switch (choice) {
157             case 1:
158                 infixToPostfix();
159                 break;
160             case 2:
161                 evaluatePostfix();
162                 break;
163             case 3:
164                 parenthesisMatching();
165                 break;
166             case 4:
167                 return 0;
168             default:
169                 printf("Invalid choice\n");
170         }
171     }
172 }

```

OUTPUT:

(i) infix to postfix

```

==== STACK APPLICATIONS ====
1. Infix to Postfix
2. Postfix Expression Evaluation
3. Parenthesis Matching
4. Exit
Enter your choice: 1

Enter an infix expression: A+B*C
Postfix expression: ABC*+

```

(ii) expression evaluation

```

==== STACK APPLICATIONS ====
1. Infix to Postfix
2. Postfix Expression Evaluation
3. Parenthesis Matching
4. Exit
Enter your choice: 2

Enter a postfix expression: 23*54*+
Result: 26

```

(iii) parenthesis matching using Stack.

```

===== STACK APPLICATIONS =====
1. Infix to Postfix
2. Postfix Expression Evaluation
3. Parenthesis Matching
4. Exit
Enter your choice: 3

Enter an expression: {[()]}
Parentheses are not balanced

```

2. Implement stack and queue using Array and Linked List.

CODE:

```

92 93      case 4:linkedQueue();break;
94      case 5:return 0;
95      default:printf("Invalid choice\n");
96  }
97  }
98  }

```

OUTPUT:

```

20 10
10 20
200 100
100 200

```

Post Lab exercises

1. Implementation of Circular Queue.